



Microsoft®
SQL Server®

SQL EXPLORATORY DATA ANALYSIS & ADVANCED ANALYTICS

END-TO-END PORTFOLIO PROJECT



BUILT USING MICROSOFT SQL SERVER

PROJECT MODULES



EXPLORATORY
DATA ANALYSIS



ADVANCED
SQL ANALYTICS



CUSTOMER
ANALYTICS



PRODUCT
ANALYTICS



BUSINESS
KPI ANALYSIS



SQL VIEWS
& REUSABLE
REPORTS



1 FACT TABLE
fact_sales



2 DIMENSION
TABLES
dim_customers
dim_products



**STRONG
RELATIONSHIPS**
Accurate Analysis
Better Insights

TECHNOLOGIES & CONCEPTS



MICROSOFT
SQL SERVER



T-SQL



CTEs



WINDOW
FUNCTIONS



SQL VIEWS

SAIDEEP PALLELA
DATA ANALYST



/saideepallela



github.com/saideepallela



<https://www.linkedin.com/in/saideepallela/>

SQL EXPLORATORY DATA ANALYSIS & ADVANCED ANALYTICS (SQL Project)

Author :- Saideep Pallela **Data Analyst** (SQL • Excel • Power BI)

Tools & Technologies: Microsoft SQL Server (T-SQL)

Objective:

This project analyses real-world sales transaction data to uncover customer behaviour, product performance, and business trends using SQL Server.

The focus is on transforming raw transactional data into business-ready insights by applying Exploratory Data Analysis (EDA), advanced SQL techniques, analytical reporting, customer and product segmentation, KPI development, and reusable SQL Views for Business Intelligence and decision-making

Data Preparation & Setup:

Database Setup:-

A dedicated database **DataWarehouseAnalytics** was created to store and manage the dataset for analytical processing.

Schema Design:-

A **gold schema** was established to represent the final curated layer used for analysis and reporting. This ensures clear separation between raw data and analysis-ready data.

Data Ingestion:

The dataset was sourced from three CSV files and imported into SQL Server using the flat file import method. Each file was loaded as a structured table within the gold schema.

Tables Created :-

gold.dim_customers – Stores customer-level attributes

gold.dim_products – Stores product-related information

gold.fact_sales – Stores transactional sales data



<https://www.linkedin.com/in/saideeppallela/>

Data Model Overview

Table Name :- gold.dim_customers

Column Name	Description
customer_key	Surrogate key for each customer (unique identifier in warehouse)
customer_id	Original customer ID from source system
customer_number	Unique customer number/code
first_name	First name of the customer
last_name	Last name of the customer
country	Country of the customer
marital_status	Marital status (Single/Married)
gender	Gender of the customer
birthdate	Date of birth of the customer
create_date	Record creation date in the system

Customer Dimension (dim_customers)

The **dim_customers** table contains descriptive attributes related to each customer. It is used to enhance transactional data by providing demographic and identification details. This enables customer-level analysis such as segmentation, behavioral insights, and identification of high-value customers. The table acts as a supporting dimension and is joined with the fact table to derive meaningful business insights.



Table Name: gold.dim_products

Column Name	Description
product_key	Surrogate key for each product (unique identifier in warehouse)
product_id	Original product ID from source system
product_number	Unique product code
product_name	Name of the product
category_id	Category identifier
category	Product category (e.g., Bikes, Components)
subcategory	Product subcategory
maintenance	Indicates if product requires maintenance (1 = Yes, 0 = No)
cost	Cost of the product
product_line	Product line classification (e.g., Road, Mountain)
start_date	Date when product became available

Product Dimension (dim_products)

The **dim_products** table contains detailed information about each product, including category, subcategory, and product classification. It is used to analyze product performance across different segments and identify revenue-driving products. This table enables category-wise analysis, comparison of product lines, and evaluation of product contribution to overall sales. It acts as a dimension table that supports joins with transactional data for deeper business insights.



Table Name: fact_sales

Column Name	Description
order_number	Unique identifier for each order
product_key	Foreign key linking to dim_products
customer_key	Foreign key linking to dim_customers
order_date	Date when order was placed
shipping_date	Date when order was shipped
due_date	Expected delivery date
sales_amount	Total revenue generated from the sale
quantity	Number of units sold
price	Price per unit

Sales Fact Table (fact_sales)

The **fact_sales** table captures transactional data and serves as the central table in the data model. It records key metrics such as sales amount, quantity, and order-level details. This table is used to perform core business analysis, including revenue calculation, sales trends, and performance evaluation. By linking with dimension tables, it enables comprehensive analysis across customers and products, supporting data-driven decision-making.



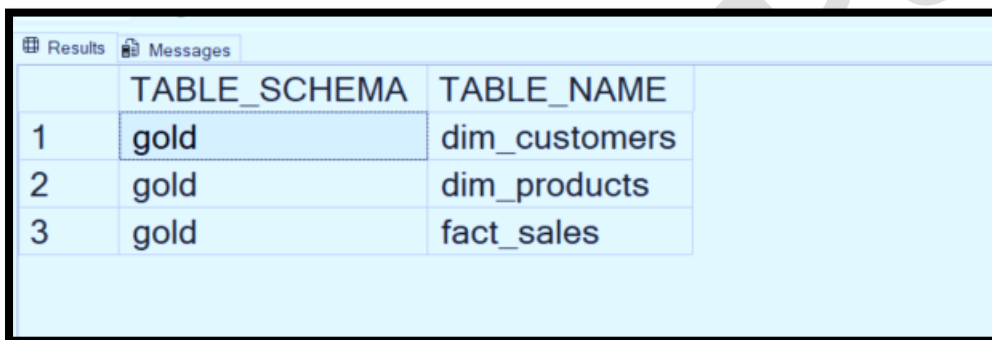
1. Exploratory Data Analysis (EDA)

1.1 Database Exploration :-

Database exploration was performed to understand the structure, quality, and completeness of the dataset before analysis.

1.1.1 Table Exploration:-

```
SELECT  
  TABLE_SCHEMA,  
  TABLE_NAME  
FROM INFORMATION_SCHEMA.TABLES  
WHERE TABLE_SCHEMA = 'gold';
```



	TABLE_SCHEMA	TABLE_NAME
1	gold	dim_customers
2	gold	dim_products
3	gold	fact_sales

1.1.2 Record Count Validation:-

```
SELECT 'fact_sales' AS table_name, COUNT(*) AS total_records FROM  
gold.fact_sales  
UNION ALL  
SELECT 'dim_customers', COUNT(*) FROM gold.dim_customers  
UNION ALL  
SELECT 'dim_products', COUNT(*) FROM gold.dim_products;
```


	table_name	total_records
1	fact_sales	60398
2	dim_customers	18484
3	dim_products	295

1.1.3 Missing Value Check:-

```
SELECT * FROM gold.fact_sales
WHERE order_number IS NULL
OR order_date IS NULL
OR sales_amount IS NULL;
```

	order_number	product_key	customer_key	order_date	shipping_date	due_date	sales_amount	quantity	price
1	SO64338	245	11820	NULL	2013-08-23	2013-08-28	35	1	35
2	SO64339	286	5323	NULL	2013-08-23	2013-08-28	25	1	25
3	SO64339	290	5323	NULL	2013-08-23	2013-08-28	4	1	4
4	SO64340	289	10522	NULL	2013-08-23	2013-08-28	5	1	5
5	SO64340	283	10522	NULL	2013-08-23	2013-08-28	30	1	30
6	SO64377	125	1374	NULL	2013-08-24	2013-08-29	2320	1	2320
7	SO64377	235	1374	NULL	2013-08-24	2013-08-29	24	1	24
8	SO64379	282	2009	NULL	2013-08-24	2013-08-29	25	1	25
9	SO64381	259	10996	NULL	2013-08-24	2013-08-29	2	1	2
10	SO64623	285	15792	NULL	2013-08-27	2013-09-01	21	1	21
11	SO64624	259	16675	NULL	2013-08-27	2013-09-01	2	1	2
12	SO64625	105	17337	NULL	2013-08-27	2013-09-01	9	1	9

1.1.4 Date Range Exploration :-

```
SELECT
MIN(order_date) AS start_date,
MAX(order_date) AS end_date
FROM gold.fact_sales;
```

	start_date	end_date
1	2010-12-29	2014-01-28

1.1.5 Key Observations:-



<https://www.linkedin.com/in/saideeppallela/>

The dataset contains one fact table and two-dimension tables for sales analysis.

The fact_sales table stores over 60K transactional records supported by customer and product data.

Missing values were identified in the order_date column during initial validation.

The dataset covers sales data from 2010 to 2014, enabling time-based analysis

1.2 Dimensions Exploration :-

Dimensions exploration was performed to understand the descriptive attributes available in the dataset, including customer locations and product hierarchy information.

1.2.1 Customer Country Exploration:-

```
SELECT  
DISTINCT country  
FROM gold.dim_customers;
```



The screenshot shows a SQL query result in a table with two columns: an index and 'country'. The results are as follows:

	country
1	n/a
2	Germany
3	United States
4	Australia
5	United Kingdom
6	Canada
7	France

1.2.2 Product Hierarchy Exploration :-

```
SELECT  
DISTINCT category, subcategory, product_name  
FROM gold.dim_products  
ORDER BY 1,2,3;
```



	category	subcategory	product_name
1	NULL	NULL	HL Mountain Pedal
2	NULL	NULL	HL Road Pedal
3	NULL	NULL	LL Mountain Pedal
4	NULL	NULL	LL Road Pedal
5	NULL	NULL	ML Mountain Pedal
6	NULL	NULL	ML Road Pedal
7	NULL	NULL	Touring Pedal
8	Accessories	Bike Racks	Hitch Rack - 4-Bike
9	Accessories	Bike Stands	All-Purpose Bike Stand
10	Accessories	Bottles and Cages	Mountain Bottle Cage
11	Accessories	Bottles and Cages	Road Bottle Cage
12	Accessories	Bottles and Cages	Water Bottle - 30 oz.

1.2.3 Key Observations :-

The customer dataset includes records from multiple countries and regions. Products are structured into categories and subcategories for better organization.

Product hierarchy information can be used for category-level and product-level analysis.

1.3 Date Exploration :-

Date exploration was performed to understand the time coverage of the dataset and analyze customer age distribution.

1.3.1 Sales Date Range Exploration :-

- What is the first and last order date available in the dataset?
- How many months of sales data are available?

SELECT

```

MIN(order_date) AS first_order_date,
MAX(order_date) AS last_order_date,
DATEDIFF(MONTH, MIN(order_date), MAX(order_date)) AS
order_range_months

```



<https://www.linkedin.com/in/saideepallela/>

FROM gold.fact_sales;

	first_order_date	last_order_date	order_range_months
1	2010-12-29	2014-01-28	37

1.3.2 Customer Age Exploration:-

-- What is the age range of customers available in the dataset?

SELECT

MIN(birthdate) **AS** oldest_birthdate,
DATEDIFF(**YEAR**, **MIN**(birthdate), **GETDATE**()) **AS** oldest_age,
MAX(birthdate) **AS** youngest_birthdate,
DATEDIFF(**YEAR**, **MAX**(birthdate), **GETDATE**()) **AS** youngest_age
FROM gold.dim_customers;

	oldest_birthdate	oldest_age	youngest_birthdate	youngest_age
1	1916-02-10	110	1986-06-25	40

1.3.3 Key Observations :-

The sales dataset covers multiple months of transactional data.

Customer birthdate information is available for age-based analysis.



The dataset includes customers across different age groups.

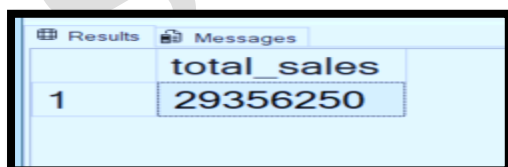
1.4 Measure Exploration :-

Measures exploration was performed to understand the key business metrics available in the dataset, including sales, quantity, orders, products, and customer-related measures.

1.4.1 Total Sales Analysis:-

---What is the total sales amount generated?

```
SELECT  
    SUM(sales_amount) AS total_sales  
FROM gold.fact_sales;
```



A screenshot of a SQL query result window. The window has two tabs: 'Results' and 'Messages'. The 'Results' tab is active, showing a table with one column named 'total_sales' and one row with the value '29356250'.

	total_sales
1	29356250

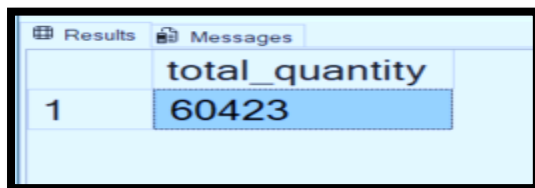
1.4.2 Total Quantity Sold

-- How many items were sold?



<https://www.linkedin.com/in/saideeppallela/>

```
SELECT  
    SUM(quantity) AS total_quantity  
FROM gold.fact_sales;
```

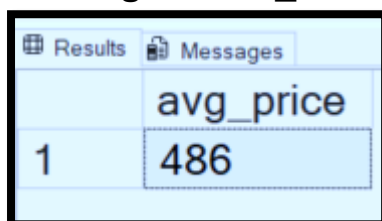


	total_quantity
1	60423

1.4.3 Average Selling Price Analysis

---- What is the average selling price?

```
SELECT  
    AVG(price) AS avg_price  
FROM gold.fact_sales;
```

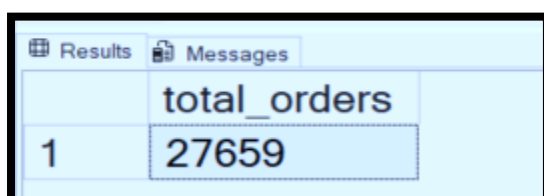


	avg_price
1	486

1.4.4 Total Orders Analysis

-- What is the total number of orders?

```
SELECT  
    COUNT(DISTINCT order_number) AS total_orders  
FROM gold.fact_sales;
```



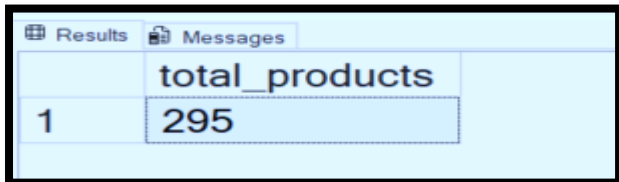
	total_orders
1	27659



1.4.5 Total Products Analysis

-- What is the total number of products?

```
SELECT  
  COUNT(DISTINCT product_name) AS total_products  
FROM gold.dim_products;
```

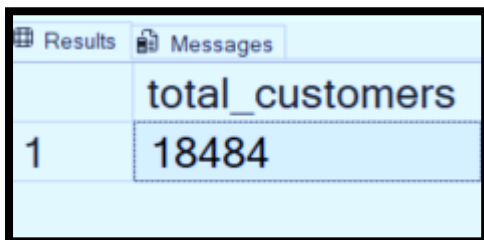


Results	
	total_products
1	295

1.4.7 Active Customers Analysis

-- How many customers have placed an order?

```
SELECT  
  COUNT(DISTINCT customer_key) AS total_customers  
FROM gold.fact_sales;
```



Results	
	total_customers
1	18484

1.4.8 Business Metrics Report

-- Generate a report showing all key business metrics

```
SELECT 'Total Sales' AS measure_name, SUM(sales_amount) AS  
measure_value FROM gold.fact_sales
```

```
UNION ALL
```

```
SELECT 'Total Quantity', SUM(quantity) FROM gold.fact_sales
```

```
UNION ALL
```

```
SELECT 'Average Price', AVG(price) FROM gold.fact_sales
```



UNION ALL

```
SELECT 'Total Orders', COUNT(DISTINCT order_number) FROM  
gold.fact_sales
```

UNION ALL

```
SELECT 'Total Products', COUNT(DISTINCT product_name) FROM  
gold.dim_products
```

UNION ALL

```
SELECT 'Total Customers', COUNT(DISTINCT customer_key) FROM  
gold.dim_customers;
```

	measure_name	measure_value
1	Total Sales	29356250
2	Total Quantity	60423
3	Average Price	486
4	Total Orders	27659
5	Total Products	295
6	Total Customers	18484

1.4.9 Key Observations

- The dataset contains key business measures related to sales, orders, products, and customers.
- Transactional data supports overall business performance analysis.
- Customer and product metrics are available for deeper analytical reporting.



1.5 Magnitude Analysis :-

Magnitude analysis was performed to understand the distribution and overall contribution of customers, products, and sales across different business dimensions.

1.5.1 Customer Distribution by Country

-- How many customers are available in each country?

```
SELECT
    country,
    COUNT(customer_key) AS total_customers
FROM gold.dim_customers
GROUP BY country
ORDER BY total_customers DESC;
```

	country	total_customers
1	United States	7482
2	Australia	3591
3	United Kingdom	1913
4	France	1810
5	Germany	1780
6	Canada	1571
7	n/a	337

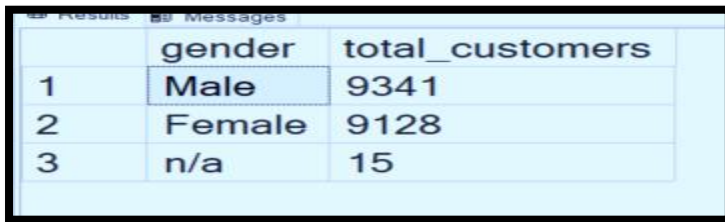
1.5.2 Customer Distribution by Gender



<https://www.linkedin.com/in/saideeppallela/>

-- How many customers are available across each gender group?

```
SELECT
gender,
COUNT(customer_key) AS total_customers
FROM gold.dim_customers
GROUP BY gender
ORDER BY total_customers DESC;
```

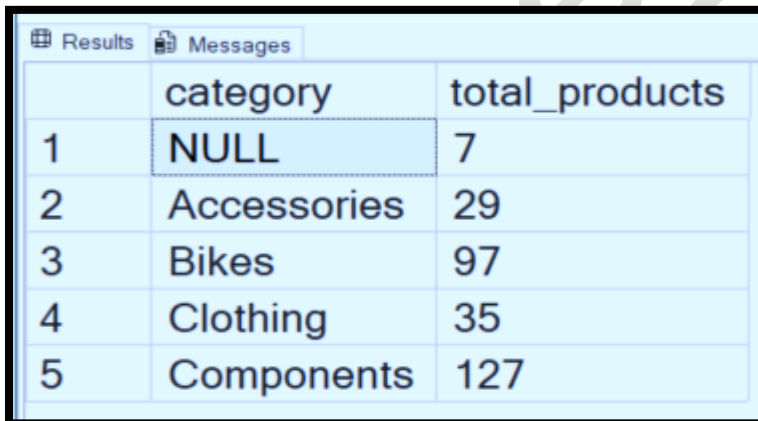


	gender	total_customers
1	Male	9341
2	Female	9128
3	n/a	15

1.5.3 Product Distribution by Category

-- How many products are available in each category?

```
SELECT
category,
COUNT(product_id) AS total_products
FROM gold.dim_products
GROUP BY category;
```



	category	total_products
1	NULL	7
2	Accessories	29
3	Bikes	97
4	Clothing	35
5	Components	127

1.5.4 Average Product Cost by Category

-- What is the average product cost in each category?

```
SELECT
category,
AVG(cost) AS avg_cost
FROM gold.dim_products
GROUP BY category
```



ORDER BY avg_cost DESC;

	category	avg_cost
1	Bikes	949
2	Components	264
3	NULL	28
4	Clothing	24
5	Accessories	13

1.5.5 Revenue Distribution by Category

-- What is the total revenue generated by each category?

SELECT

d.category,
SUM(f.sales_amount) AS total_revenue
FROM gold.fact_sales AS f
LEFT JOIN gold.dim_products AS d
ON f.product_key = d.product_key
GROUP BY d.category
ORDER BY total_revenue DESC;

	category	total_revenue
1	Bikes	28316272
2	Accessories	700262
3	Clothing	339716

1.5.6 Customer Revenue Contribution

-- How much revenue is generated by each customer?

SELECT



<https://www.linkedin.com/in/saideeppallela/>

```

d.customer_key,
d.first_name,
d.last_name,
SUM(f.sales_amount) AS total_revenue
FROM gold.fact_sales AS f
LEFT JOIN gold.dim_customers AS d
ON f.customer_key = d.customer_key
GROUP BY
d.customer_key,
d.first_name,
d.last_name
ORDER BY total_revenue DESC;

```

1.5.6 Customer Revenue Contribution query out put

	customer_key	first_name	last_name	total_revenue
1	1133	Kaitlyn	Henderson	13294
2	1302	Nichole	Nara	13294
3	1309	Margaret	He	13268
4	1132	Randall	Dominguez	13265
5	1301	Adriana	Gonzalez	13242
6	1322	Rosa	Hu	13215
7	1125	Brandi	Gill	13195
8	1308	Brad	She	13172

Query executed successfully. LK9MEEMA\SQLEXPRESS (17.0 RTM) LK9MEEMA\lenovo (51) DataWarehouseAnalytics 00:00:00 Row: 4, Col: 1 18,484 rows

1.5.7 Sold Items Distribution by Country

-- What is the distribution of sold items across countries?

```

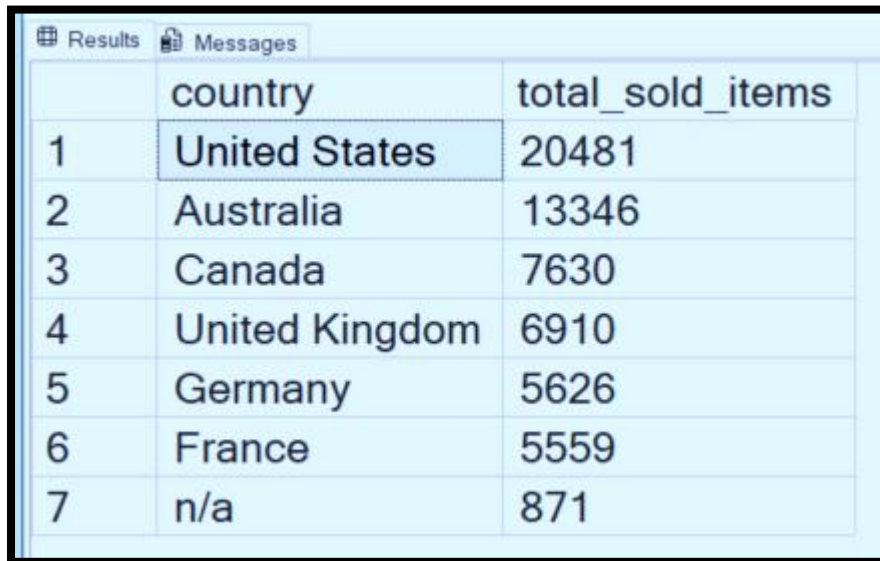
SELECT
c.country,
SUM(s.quantity) AS total_sold_items
FROM gold.fact_sales AS s
LEFT JOIN gold.dim_customers AS c
ON c.customer_key = s.customer_key
GROUP BY c.country

```



<https://www.linkedin.com/in/saideeppallela/>

ORDER BY total_sold_items DESC;



	country	total_sold_items
1	United States	20481
2	Australia	13346
3	Canada	7630
4	United Kingdom	6910
5	Germany	5626
6	France	5559
7	n/a	871

1.5.8 Key Observations

- Customer distribution varies across different countries and demographic groups.
- Product categories contribute differently to overall revenue and sales performance.
- Revenue generation is concentrated among specific customer groups and product categories.
- Sales quantity distribution differs across countries and business segments.



1.6 Ranking Analysis :-

Ranking analysis was performed to identify top-performing and low-performing products and customers based on revenue and order activity.

1.6.1 Top 5 Revenue-Generating Products

-- Which 5 products generate the highest revenue?

```
SELECT TOP 5
    p.product_name,
    SUM(f.sales_amount) AS total_revenue
FROM gold.fact_sales f
LEFT JOIN gold.dim_products p
ON p.product_key = f.product_key
GROUP BY p.product_name
ORDER BY total_revenue DESC;
```



	product_name	total_revenue
1	Mountain-200 Black- 46	1373454
2	Mountain-200 Black- 42	1363128
3	Mountain-200 Silver- 38	1339394
4	Mountain-200 Silver- 46	1301029
5	Mountain-200 Black- 38	1294854

1.6.2 Top 5 Products Using Window Function

-- Which 5 products generate the highest revenue using ranking functions?

```

SELECT *
FROM (
  SELECT
    p.product_name,
    SUM(f.sales_amount) AS total_revenue,
    ROW_NUMBER() OVER (
      ORDER BY SUM(f.sales_amount) DESC
    ) AS rank_products
  FROM gold.fact_sales f
  LEFT JOIN gold.dim_products p

```



<https://www.linkedin.com/in/saideeppallela/>


```

ON p.product_key = f.product_key
GROUP BY p.product_name
) t
WHERE rank_products <= 5;

```

	product_name	total_revenue	rank_products
1	Mountain-200 Black- 46	1373454	1
2	Mountain-200 Black- 42	1363128	2
3	Mountain-200 Silver- 38	1339394	3
4	Mountain-200 Silver- 46	1301029	4
5	Mountain-200 Black- 38	1294854	5

1.6.3 Lowest Revenue-Generating Products

-- What are the 5 worst-performing products in terms of revenue?

```

SELECT TOP 5
    p.product_name,
    SUM(f.sales_amount) AS total_revenue
FROM gold.fact_sales f
LEFT JOIN gold.dim_products p
ON p.product_key = f.product_key
GROUP BY p.product_name
ORDER BY total_revenue ASC;

```



	product_name	total_revenue
1	Racing Socks- L	2430
2	Racing Socks- M	2682
3	Patch Kit/8 Patches	6382
4	Bike Wash - Dissolver	7272
5	Touring Tire Tube	7440

1.6.4 Top Revenue-Generating Customers

-- Which 10 customers generated the highest revenue?

```

SELECT TOP 10
  c.customer_key,
  c.first_name,
  c.last_name,
  SUM(f.sales_amount) AS total_revenue
FROM gold.fact_sales f
LEFT JOIN gold.dim_customers c
ON c.customer_key = f.customer_key
GROUP BY
  c.customer_key,
  c.first_name,
  c.last_name
ORDER BY total_revenue DESC;

```

	customer_key	first_name	last_name	total_revenue
1	1133	Kaitlyn	Henderson	13294
2	1302	Nichole	Nara	13294
3	1309	Margaret	He	13268
4	1132	Randall	Dominguez	13265
5	1301	Adriana	Gonzalez	13242
6	1322	Rosa	Hu	13215
7	1125	Brandi	Gill	13195
8	1308	Brad	She	13172
9	1297	Francisco	Sara	13164
10	434	Maurice	Shan	12914

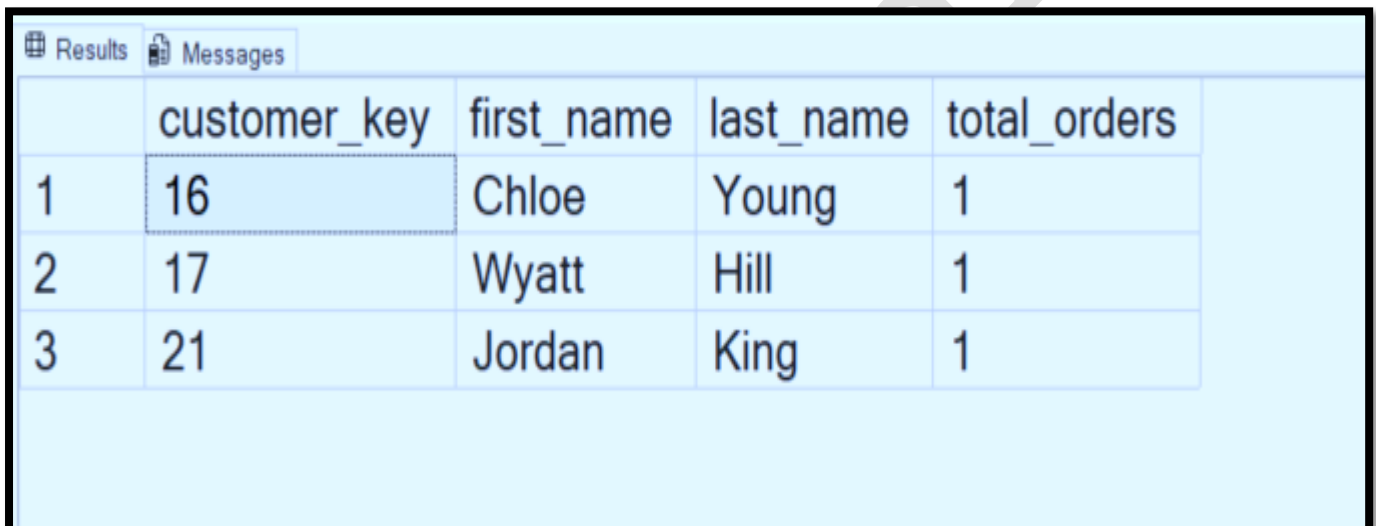
1.6.5 Customers with Lowest Orders



<https://www.linkedin.com/in/saideeppallela/>

-- Which 3 customers placed the fewest orders?

```
SELECT TOP 3
  c.customer_key,
  c.first_name,
  c.last_name,
  COUNT(DISTINCT f.order_number) AS total_orders
FROM gold.fact_sales f
LEFT JOIN gold.dim_customers c
ON c.customer_key = f.customer_key
GROUP BY
  c.customer_key,
  c.first_name,
  c.last_name
ORDER BY total_orders ASC;
```



The screenshot shows a SQL query results window with two tabs: 'Results' and 'Messages'. The 'Results' tab is active, displaying a table with 5 columns: 'customer_key', 'first_name', 'last_name', and 'total_orders'. The table contains 3 rows of data, representing the top 3 customers with the fewest orders. The first row shows customer_key 16, first_name Chloe, last_name Young, and total_orders 1. The second row shows customer_key 17, first_name Wyatt, last_name Hill, and total_orders 1. The third row shows customer_key 21, first_name Jordan, last_name King, and total_orders 1.

	customer_key	first_name	last_name	total_orders
1	16	Chloe	Young	1
2	17	Wyatt	Hill	1
3	21	Jordan	King	1

1.6.6 Key Observations

- Revenue generation is concentrated among a limited number of products and customers.
- Some products contribute significantly less to overall business revenue.
- Ranking functions help identify top-performing business entities efficiently.
- Customer order behaviour varies across different customer segments.

2. Advanced Business Analysis



<https://www.linkedin.com/in/saideeppallela/>

Advanced business analysis was performed to gain deeper insights into sales performance and business trends. Advanced SQL techniques, including CTEs and window functions, were used to analyze performance, identify patterns, and support data-driven decision-making.

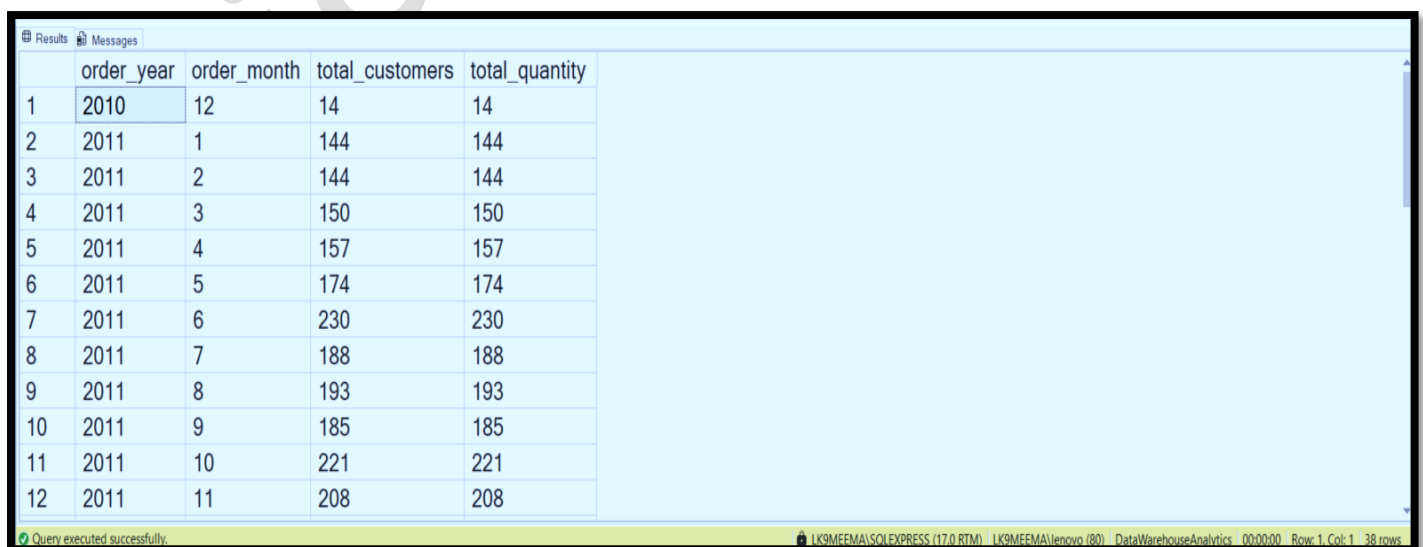
2.1 Change Over Time Analysis

Change over time analysis was performed to evaluate how key business measures evolve across different time periods. This analysis helps identify trends, growth patterns, and seasonality in business performance.

2.1.1 Monthly Trends Analysis

-- How do customer activity and sales quantity change over time monthly basis?

```
SELECT
    YEAR(order_date) AS order_year,
    MONTH(order_date) AS order_month,
    COUNT(DISTINCT customer_key) AS total_customers,
    SUM(quantity) AS total_quantity
FROM gold.fact_sales
WHERE order_date IS NOT NULL
GROUP BY YEAR(order_date), MONTH(order_date)
ORDER BY YEAR(order_date), MONTH(order_date);
```



	order_year	order_month	total_customers	total_quantity
1	2010	12	14	14
2	2011	1	144	144
3	2011	2	144	144
4	2011	3	150	150
5	2011	4	157	157
6	2011	5	174	174
7	2011	6	230	230
8	2011	7	188	188
9	2011	8	193	193
10	2011	9	185	185
11	2011	10	221	221
12	2011	11	208	208



<https://www.linkedin.com/in/saideepallela/>

2.1.2 Monthly Trends Using DATETRUNC

-- How do customer activity and sales quantity evolve over time using monthly date aggregation?

```
SELECT
    DATETRUNC(MONTH, order_date) AS order_date,
    COUNT(DISTINCT customer_key) AS total_customers,
    SUM(quantity) AS total_quantity
FROM gold.fact_sales
WHERE order_date IS NOT NULL
GROUP BY DATETRUNC(MONTH, order_date)
ORDER BY DATETRUNC(MONTH, order_date);
```



	order_date	total_customers	total_quantity
1	2010-12-01	14	14
2	2011-01-01	144	144
3	2011-02-01	144	144
4	2011-03-01	150	150
5	2011-04-01	157	157
6	2011-05-01	174	174
7	2011-06-01	230	230
8	2011-07-01	188	188
9	2011-08-01	193	193
10	2011-09-01	185	185
11	2011-10-01	221	221
12	2011-11-01	208	208

Query executed successfully. LK9MEEMA\SQLEXPRESS (17.0 RTM) LK9MEEMA\lenovo (80) DataWarehouseAnalytics 00:00:00 Row: 1, Col: 1 38 rows

2.1.3 Monthly Trends Using Formatted Dates

-- How can monthly business trends be presented in a reporting-friendly format?

```
SELECT
    FORMAT(order_date, 'yyyy-MMM') AS order_date,
    COUNT(DISTINCT customer_key) AS total_customers,
    SUM(quantity) AS total_quantity
FROM gold.fact_sales
WHERE order_date IS NOT NULL
```



<https://www.linkedin.com/in/saideeppallela/>

GROUP BY FORMAT(order_date, 'yyyy-MMM')
ORDER BY FORMAT(order_date, 'yyyy-MMM');

	order_date	total_customers	total_quantity
1	2010-Dec	14	14
2	2011-Apr	157	157
3	2011-Aug	193	193
4	2011-Dec	222	222
5	2011-Feb	144	144
6	2011-Jan	144	144
7	2011-Jul	188	188
8	2011-Jun	230	230
9	2011-Mar	150	150
10	2011-May	174	174
11	2011-Nov	208	208
12	2011-Oct	221	221

Query executed successfully. LK9MEEMA\SQLEXPRESS (17.0 RTM) LK9MEEMA\lenovo (80) DataWarehouseAnalytics 000001 Row: 1, Col: 1 38 rows

2.1.4 Key Observations

- Customer activity and sales quantity fluctuate across different months.
- Time-based aggregation helps identify trends and seasonal patterns in business performance.
- Different SQL date functions can be used to support analytical and reporting requirements.

2.2 Cumulative Analysis

Cumulative analysis was performed to analyze business performance over time by calculating running totals and moving averages. This analysis helps track business growth, monitor performance trends, and understand long-term patterns in sales data.

2.2.1 Running Total and Moving Average Analysis



<https://www.linkedin.com/in/saideeppallela/>

-- What is the cumulative sales trend and moving average price over time?

```
SELECT
    order_date,
    total_sales,
    SUM(total_sales) OVER (ORDER BY order_date) AS running_total_sales,
    AVG(avg_price) OVER (ORDER BY order_date) AS moving_average_price
FROM
(
    SELECT
        DATETRUNC(YEAR, order_date) AS order_date,
        SUM(sales_amount) AS total_sales,
        AVG(price) AS avg_price
    FROM gold.fact_sales
    WHERE order_date IS NOT NULL
    GROUP BY DATETRUNC(YEAR, order_date)
) AS yearly_sales;
```



The screenshot shows a SQL query result in a table with 5 columns: an index, order_date, total_sales, running_total_sales, and moving_average_price. The data spans from 2010 to 2014. The running_total_sales column shows a cumulative increase over the years, while the moving_average_price column shows a relatively stable average price around 3100.

	order_date	total_sales	running_total_sales	moving_average_price
1	2010-01-01	43419	43419	3101
2	2011-01-01	7075088	7118507	3146
3	2012-01-01	5842231	12960738	2670
4	2013-01-01	16344878	29305616	2080
5	2014-01-01	45642	29351258	1668

2.2.2 Key Observations

- Running totals help track cumulative business growth over time.
- Moving averages smooth short-term fluctuations and highlight long-term trends.
- Window functions enable advanced time-based analysis without complex joins.



2.3 Performance Analysis

Performance analysis was conducted to evaluate how individual products perform relative to the overall product portfolio. Rather than only measuring total sales, this analysis compares each product against key business metrics such as the average performance and the highest-performing product. These benchmarks help identify top-performing products, underperforming products, and opportunities for business improvement.

2.3.1 Yearly Product Performance Analysis

-- Analyze the yearly performance of products by comparing each product's sales to both its average sales performance and the previous year's sales.

WITH yearly_sales **AS**

```
(  
  SELECT  
    YEAR(f.order_date) AS order_year,  
    p.product_name,  
    SUM(f.sales_amount) AS current_sales  
  FROM gold.fact_sales f  
  JOIN gold.dim_products p  
    ON f.product_key = p.product_key  
  WHERE f.order_date IS NOT NULL  
  GROUP BY YEAR(f.order_date), p.product_name  
)
```

```
SELECT  
  order_year,  
  product_name,  
  current_sales,  
  AVG(current_sales) OVER(PARTITION BY product_name) AS  
  average_sales,  
  current_sales-AVG(current_sales) OVER(PARTITION BY product_name) AS  
  average_difference,  
  CASE  
    WHEN current_sales>AVG(current_sales) OVER(PARTITION BY  
product_name) THEN 'Above Avg'
```




```

    WHEN current_sales < AVG(current_sales) OVER(PARTITION BY
product_name) THEN 'Below Avg'
    ELSE 'Average'
END AS average_status,
LAG(current_sales) OVER(PARTITION BY product_name ORDER BY
order_year) AS previous_year_sales,
current_sales - LAG(current_sales) OVER(PARTITION BY product_name
ORDER BY order_year) AS year_over_year_difference,
CASE
    WHEN current_sales > LAG(current_sales) OVER(PARTITION BY
product_name ORDER BY order_year) THEN 'Increase'
    WHEN current_sales < LAG(current_sales) OVER(PARTITION BY
product_name ORDER BY order_year) THEN 'Decrease'
    ELSE 'No Change'
END AS year_over_year_status
FROM yearly_sales
ORDER BY product_name, order_year;

```

	product_name	current_sales	average_sales	average_difference	average_status	previous_year_sales	year_over_year_difference	year_over_year_status
133	-250 Black- 44	6546	209461	-202915	Below Avg	NULL	NULL	No Change
134	-250 Black- 44	279818	209461	70357	Above Avg	6546	273272	Increase
135	-250 Black- 44	342020	209461	132559	Above Avg	279818	62202	Increase
136	-250 Black- 48	310105	345606	-35501	Below Avg	NULL	NULL	No Change
137	-250 Black- 48	381108	345606	35502	Above Avg	310105	71003	Increase
138	-250 Black- 52	4364	244808	-240444	Below Avg	NULL	NULL	No Change
139	-250 Black- 52	378269	244808	133461	Above Avg	4364	373905	Increase
140	-250 Black- 52	351792	244808	106984	Above Avg	378269	-26477	Decrease
141	-250 Black- 58	2182	207342	-205160	Below Avg	NULL	NULL	No Change
142	-250 Black- 58	316912	207342	109570	Above Avg	2182	314730	Increase
143	-250 Black- 58	302932	207342	95590	Above Avg	316912	-13980	Decrease
144	-250 Red- 44	4886	175896	-171010	Below Avg	NULL	NULL	No Change
145	-250 Red- 44	346906	175896	171010	Above Avg	4886	342020	Increase
146	-250 Red- 48	7329	197883	-190554	Below Avg	NULL	NULL	No Change
147	-250 Red- 48	388437	197883	190554	Above Avg	7329	381108	Increase
148	-250 Red- 52	2443	162459	-160016	Below Avg	NULL	NULL	No Change

2.3.2 Key Observations

- Yearly sales were benchmarked against each product's historical average to evaluate overall performance.
- Products performing above their average sales indicate consistent business growth and strong market demand.
- Year-over-year comparisons help identify sales increases, decreases, or stable performance across different years.



<https://www.linkedin.com/in/saideepallela/>

- The LAG() window function enables efficient year-over-year analysis without requiring complex self-joins.
- Combining historical averages with previous-year comparisons provides a more comprehensive evaluation of product performance than relying solely on total sales.

2.4 Part-to-Whole Analysis

Part-to-whole analysis was performed to determine how much each product category contributes to the organization's total sales. Rather than focusing solely on individual sales figures, this analysis calculates the percentage share of overall revenue generated by each product category. These insights help identify the primary revenue-driving categories, understand the distribution of sales across the product portfolio, and support data-driven decisions related to inventory management, product strategy, and resource allocation.

2.4.1 Yearly Product Performance Analysis

-- Which product categories contribute the most to total sales?

```
WITH category_sales AS
(
    SELECT
        p.category,
        SUM(f.sales_amount) AS total_sales
    FROM gold.fact_sales f
    JOIN gold.dim_products p
    ON f.product_key=p.product_key
    GROUP BY p.category
)
SELECT
    category,
    total_sales,
    SUM(total_sales) OVER() AS overall_sales,
    CONCAT(ROUND(CAST(total_sales AS FLOAT)
/SUM(total_sales) OVER()*100,2),'%') AS percentage_of_total
FROM category_sales
ORDER BY total_sales DESC;
```



	category	total_sales	overall_sales	percentage_of_total
1	Bikes	28316272	29356250	96.46%
2	Accessories	700262	29356250	2.39%
3	Clothing	339716	29356250	1.16%

2.4.2 Key Observations

- Total sales were aggregated at the product category level to evaluate each category's overall business contribution.
- Percentage contribution provides a clear understanding of how revenue is distributed across different product categories.
- Categories with the highest contribution represent the primary revenue drivers and can be prioritized for marketing, inventory planning, and business growth initiatives.
- Categories with lower contribution percentages may require further analysis to identify opportunities for performance improvement or portfolio optimization.
- SQL window functions simplify part-to-whole analysis by calculating the overall sales total within the same query, eliminating the need for additional joins or nested queries.

Start from sec 17 173. Data Segmentation already sql file saved in advanced analytics 11

2.5 Data Segmentation:-



<https://www.linkedin.com/in/saideeppallela/>

Data segmentation was performed to group business data into meaningful categories based on predefined value ranges. Instead of analyzing individual records, segmentation organizes products or customers into logical groups according to specific measures such as sales, revenue, or age. This approach simplifies data analysis, uncovers hidden patterns, and enables businesses to develop targeted strategies for different segments.

2.5.1 Product Segmentation by Cost Range

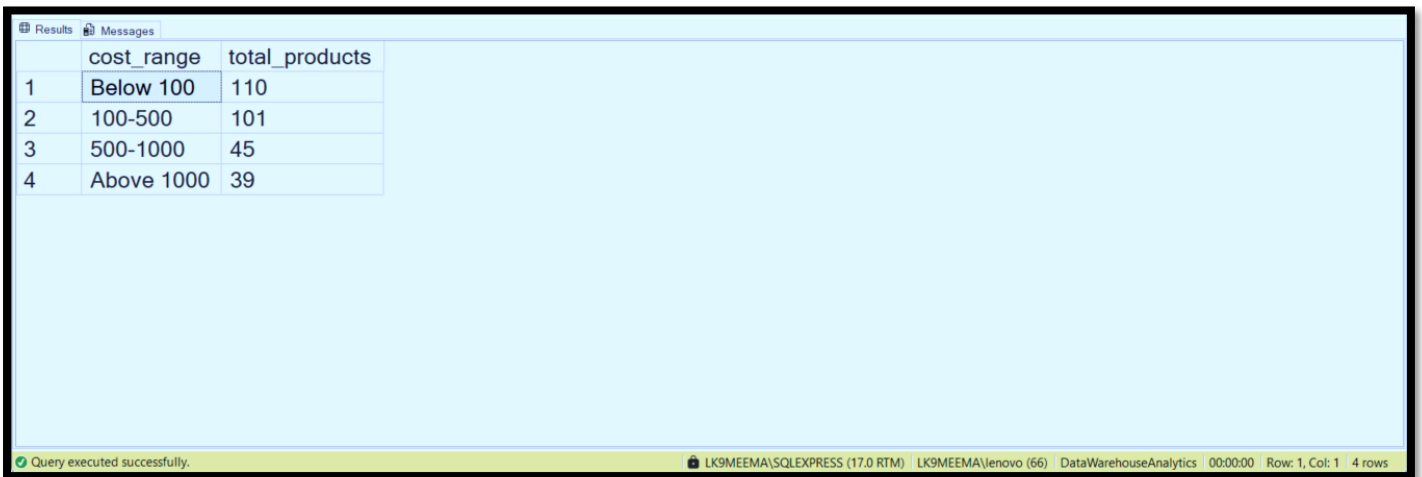
-- How can products be segmented into different cost ranges, and how many products belong to each segment?

WITH product_segments **AS**

```
(  
  SELECT  
    product_key,  
    product_name,  
    cost,  
    CASE  
      WHEN cost < 100 THEN 'Below 100'  
      WHEN cost BETWEEN 100 AND 500 THEN '100-500'  
      WHEN cost BETWEEN 500 AND 1000 THEN '500-1000'  
      ELSE 'Above 1000'  
    END AS cost_range  
  FROM gold.dim_products  
)
```

```
SELECT  
  cost_range,  
  COUNT(product_key) AS total_products  
FROM product_segments  
GROUP BY cost_range  
ORDER BY total_products DESC;
```





The screenshot shows a SQL query results window with a table containing four rows of data. The table has two columns: 'cost_range' and 'total_products'. The rows represent different cost categories and the number of products in each category.

	cost_range	total_products
1	Below 100	110
2	100-500	101
3	500-1000	45
4	Above 1000	39

The status bar at the bottom indicates: Query executed successfully. LK9MEEMA\SQLEXPRESS (17.0 RTM) LK9MEEMA\lenovo (66) DataWarehouseAnalytics 00:00:00 Row: 1, Col: 1 4 rows

2.5.2 Key Observations

Products were categorized into four predefined cost ranges based on their unit cost.

The CASE expression was used to transform continuous cost values into meaningful business segments.

The analysis provides a clear view of how products are distributed across different cost categories.

Grouping products by cost range helps identify whether the product portfolio is concentrated in budget, mid-range, or premium segments.

Cost-based segmentation supports pricing analysis, inventory planning, and product portfolio management



2.5.3 Customer Segmentation Based on Spending Behavior

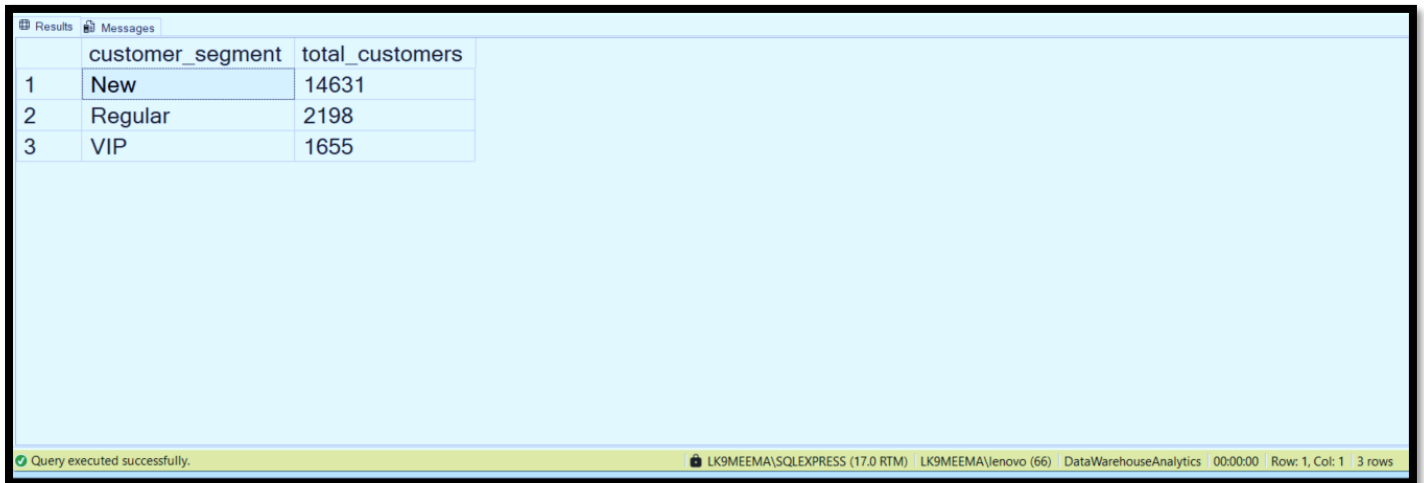
How can customers be segmented based on their spending behavior and purchase history, and how many customers belong to each segment?

WITH customer_spending **AS**

```
(  
  SELECT  
    c.customer_key,  
    SUM(f.sales_amount) AS total_spending,  
    MIN(order_date) AS first_order,  
    MAX(order_date) AS last_order,  
    DATEDIFF(month, MIN(order_date), MAX(order_date)) AS lifespan  
FROM gold.fact_sales f  
LEFT JOIN gold.dim_customers c  
  ON f.customer_key = c.customer_key  
GROUP BY c.customer_key  
)
```

```
SELECT  
  customer_segment,  
  COUNT(customer_key) AS total_customers  
FROM  
(  
  SELECT  
    customer_key,  
    CASE  
      WHEN lifespan >= 12 AND total_spending > 5000 THEN 'VIP'  
      WHEN lifespan >= 12 AND total_spending <= 5000 THEN 'Regular'  
      ELSE 'New'  
    END AS customer_segment  
FROM customer_spending  
) t  
GROUP BY customer_segment  
ORDER BY total_customers DESC;
```





The screenshot shows a SQL query results window with a table containing customer segmentation data. The table has two columns: 'customer_segment' and 'total_customers'. There are three rows of data: 'New' with 14631 customers, 'Regular' with 2198 customers, and 'VIP' with 1655 customers. The window also shows a status bar at the bottom indicating the query was executed successfully.

	customer_segment	total_customers
1	New	14631
2	Regular	2198
3	VIP	1655

2.5.2 Key Observations

Customers were segmented into VIP, Regular, and new categories based on their purchase history and total spending.

The segmentation combines customer lifespan and spending behavior, providing a more comprehensive evaluation of customer value than using a single metric.

VIP customers represent long-term, high-value customers who contribute significantly to overall business revenue and should be prioritized for retention and loyalty programs.

Regular customers demonstrate consistent purchasing behavior but have lower overall spending, presenting opportunities for upselling and cross-selling initiatives.

New customers have a shorter purchase history and may require targeted onboarding, personalized promotions, and engagement strategies to encourage repeat purchases and long-term loyalty.

This segmentation helps businesses understand customer value, optimize marketing efforts, improve customer retention, and allocate resources more effectively based on customer behavior.



2.6 Customer Report:-

A customer report was developed to consolidate customer demographics, purchasing behavior, and key performance indicators into a single analytical view. The report retrieves essential customer information, classifies customers into meaningful business segments and age groups, calculates customer-level metrics, and generates KPIs that help evaluate customer value, engagement, and purchasing patterns. To improve reusability and support Business Intelligence reporting, the final output was created as a SQL Server View.

2.6.1 Customer Report:-

Create a Customer Report that:

- 1. Retrieves essential customer information (such as customer name, age, and transaction details).**
- 2. Segments customers into VIP, Regular, and New categories, and classifies them into age groups.**
- 3. Calculates customer-level metrics including:**
 - Total Orders**
 - Total Sales**
 - Total Quantity Purchased**
 - Total Products Purchased**
 - Customer Lifespan (in months)**
- 4. Computes the following KPIs:**
 - Recency (months since last order)**
 - Average Order Value**
 - Average Monthly Spend**




```
CREATE VIEW gold.report_customers AS
WITH base_query AS
/*
```

1) Base Query: Retrieves core customer and sales data for further analysis.

```
*/
(
  SELECT
    f.order_number,
    f.product_key,
    f.order_date,
    f.sales_amount,
    f.quantity,
    c.customer_key,
    c.customer_number,
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
    DATEDIFF(YEAR, c.birthdate, GETDATE()) AS age
  FROM gold.fact_sales f
  LEFT JOIN gold.dim_customers c
    ON c.customer_key = f.customer_key
  WHERE order_date IS NOT NULL
)
, customer_aggregation AS (
/*
```

2) Customer Aggregations: Summarizes key metrics at the customer level

```
*/
SELECT
  customer_key,
  customer_number,
  customer_name,
  age,
  COUNT(DISTINCT order_number) AS total_orders,
  SUM(sales_amount) AS total_sales,
  SUM(quantity) AS total_quantity,
  COUNT(DISTINCT product_key) AS total_products,
  MAX(order_date) AS last_order_date,
  DATEDIFF(MONTH, MIN(order_date), MAX(order_date)) AS lifespan
```



```

FROM base_query
GROUP BY
    customer_key,
    customer_number,
    customer_name,
    age
)
SELECT
    customer_key,
    customer_number,
    customer_name,
    age,
    CASE
        WHEN age < 20 THEN 'Under 20'
        WHEN age BETWEEN 20 AND 29 THEN '20-29'
        WHEN age BETWEEN 30 AND 39 THEN '30-39'
        WHEN age BETWEEN 40 AND 49 THEN '40-49'
        ELSE '50 and above'
    END AS age_group,
    CASE
        WHEN lifespan >= 12 AND total_sales > 5000 THEN 'VIP'
        WHEN lifespan >= 12 AND total_sales <= 5000 THEN 'Regular'
        ELSE 'New'
    END AS customer_segment,
    last_order_date,
    DATEDIFF(MONTH, last_order_date, GETDATE()) AS recency,
    total_orders,
    total_sales,
    total_quantity,
    total_products,
    lifespan,
    ---compute average order value (AVO)
    CASE
        WHEN total_orders = 0 THEN 0
        ELSE total_sales/total_orders
    END AS avg_order_value,
    ---compute average monthly spend
    CASE
        WHEN lifespan = 0 then total_sales
        ELSE total_sales / lifespan
    END AS avg_monthly_spend
FROM Customer_aggregation;

```



SELECT * FROM gold.report_customers

	customer_key	customer_number	customer_name	age	age_group	customer_segment	last_order_date	recency	total_orders	total_sales	total_quantity	total_products	lifespan	avg_order_value	avg_monthly_spend
45	45	AW00011044	Adam Flores	72	50 and abo...	VIP	2013-11-08	152	3	6515	7	7	33	2171	197
46	46	AW00011045	Leonard Nara	71	50 and abo...	New	2013-10-29	153	2	115	4	4	6	57	19
47	47	AW00011046	Christine Yuan	71	50 and abo...	VIP	2013-11-12	152	3	6616	6	6	33	2205	200
48	48	AW00011047	Jaclyn Lu	71	50 and abo...	VIP	2013-11-19	152	3	6582	4	4	33	2194	199
49	49	AW00011048	Jeremy Powell	70	50 and abo...	VIP	2013-12-25	151	3	6498	8	8	34	2166	191
50	50	AW00011049	Carol Rai	40	40-49	New	2013-10-29	153	2	81	5	5	7	40	11
51	51	AW00011050	Alan Zheng	69	50 and abo...	VIP	2013-11-19	152	3	5007	6	6	33	1669	151
52	52	AW00011051	Daniel Johnson	69	50 and abo...	New	2014-01-01	150	2	124	5	5	1	62	124
53	53	AW00011052	Heidi Lopez	69	50 and abo...	VIP	2013-05-27	158	3	8131	7	7	27	2710	301
54	54	AW00011053	Ana Price	40	40-49	New	2013-01-17	162	1	2330	2	2	0	2330	2330
55	55	AW00011054	Deanna Munoz	69	50 and abo...	VIP	2013-05-30	158	3	8223	6	6	27	2741	304
56	56	AW00011055	Gilbert Rajc	69	50 and abo...	VIP	2013-06-07	157	3	8168	8	8	29	2722	281
57	57	AW00011056	Michele Nath	68	50 and abo...	VIP	2013-05-31	158	3	8207	8	8	27	2735	303
58	58	AW00011057	Carl Andersen	56	50 and abo...	VIP	2013-05-21	158	3	8088	4	4	27	2696	299
59	59	AW00011058	Marc Diaz	61	50 and abo...	VIP	2013-06-04	157	3	8248	6	6	27	2749	305
60	60	AW00011059	Ashlee Andersen	67	50 and abo...	New	2013-03-16	160	3	117	7	6	1	39	117
61	61	AW00011060	Jon Zhou	67	50 and abo...	VIP	2013-06-25	157	3	8153	6	6	27	2717	301
62	62	AW00011061	Todd Gao	50	50 and abo...	VIP	2013-06-14	157	3	8129	6	6	27	2709	301
63	63	AW00011062	Noah Powell	45	40-49	New	2013-01-07	162	1	2352	3	3	0	2352	2352
64	64	AW00011063	Angela Murphy	46	40-49	New	2013-01-12	162	1	2373	5	5	0	2373	2373
65	65	AW00011064	Chase Reed	45	40-49	New	2013-01-25	162	1	2305	2	2	0	2305	2305
66	66	AW00011065	Jessica Hender...	47	40-49	New	2014-01-10	150	2	202	5	4	11	101	18
67	67	AW00011066	Grace Butler	47	40-49	New	2013-09-19	154	2	81	5	5	4	40	20
68	68	AW00011067	Caleb Carter	44	40-49	New	2014-01-12	150	2	105	6	6	5	52	21
69	69	AW00011068	Tiffany Liang	60	50 and abo...	New	2013-10-08	153	3	113	9	8	6	37	18
70	70	AW00011069	Carolyn Navarro	60	50 and abo...	VIP	2013-06-04	157	3	8104	3	3	26	2701	311
71	71	AW00011070	Willie Raji	66	50 and abo...	VIP	2013-06-26	157	3	8165	10	9	26	2721	314
72	72	AW00011071	Linda Serrano	66	50 and abo...	New	2013-12-01	151	3	110	7	6	9	36	12
73	73	AW00011072	Casey Luo	66	50 and abo...	VIP	2013-06-15	157	3	8194	7	7	27	2731	303

2.6.2 Key Observations

- Customer demographic and transaction data were consolidated into a single analytical report.
- Customers were segmented into VIP, Regular, and new categories based on their purchase history and total spending.
- Age groups were created to support demographic and customer profiling analysis.
- Customer-level metrics such as total orders, sales, quantity purchased, products purchased, and lifespan provide a comprehensive view of purchasing behavior.
- Key Performance Indicators (KPIs), including Recency, Average Order Value (AOV), and Average Monthly Spend, help measure customer engagement and business value.



<https://www.linkedin.com/in/saideeppallela/>

- Creating the report as a SQL Server View improves reusability and enables seamless integration with reporting tools such as Power BI and Excel.

2.7 Product Report:-

The Product Report was developed to create a centralized analytical view that combines product information, sales performance, and key business metrics. By integrating product master data with transactional sales records, the report provides a comprehensive understanding of product performance, customer reach, and revenue generation. The final report is implemented as a reusable SQL Server View, enabling consistent reporting and seamless integration with Business Intelligence tools such as Power BI and Excel.

2.7.1 Product Report:-

Create a Product Report that:

- 1. Retrieves essential product information (such as product name, category, subcategory, and cost).**
- 2. Segments products into High-Performers, Mid-Range, and Low-Performers based on revenue.**
- 3. Calculates product-level metrics including:**
 - Total Orders**
 - Total Sales**
 - Total Quantity Sold**
 - Total Unique Customers**
 - Product Lifespan (in months)**
- 4. Computes the following KPIs:**
 - Recency (months since last sale)**
 - Average Order Revenue (AOR)**



- Average Monthly Revenue

CREATE VIEW gold.report_products **AS**

WITH base_query **AS**

```
(  
  /*  
  -----  
  1) Base Query: Retrieves core columns from fact_sales and dim_products  
  -----  
  */  
  SELECT  
    f.order_number,  
    f.order_date,  
    f.customer_key,  
    f.sales_amount,  
    f.quantity,  
    p.product_key,  
    p.product_name,  
    p.category,  
    p.subcategory,  
    p.cost  
  FROM gold.fact_sales f  
  LEFT JOIN gold.dim_products p  
    ON f.product_key = p.product_key  
  WHERE order_date IS NOT NULL    -- Only consider valid sales dates  
)
```

product_aggregations **AS**

```
(  
  /*  
  -----  
  2) Product Aggregations: Summarizes key metrics at the product level  
  -----  
  */  
  SELECT  
    product_key,  
    product_name,  
    category,  
    subcategory,
```



```

cost,
DATEDIFF(MONTH, MIN(order_date), MAX(order_date)) AS lifespan,
MAX(order_date) AS last_sale_date,
COUNT(DISTINCT order_number) AS total_orders,
COUNT(DISTINCT customer_key) AS total_customers,
SUM(sales_amount) AS total_sales,
SUM(quantity) AS total_quantity,
ROUND(
    AVG(CAST(sales_amount AS FLOAT) / NULLIF(quantity, 0)),
    1
) AS avg_selling_price
FROM base_query
GROUP BY
    product_key,
    product_name,
    category,
    subcategory,
    cost
)

/*

```

3) Final Query: Combines all product results into one output

```

*/
SELECT
    product_key,
    product_name,
    category,
    subcategory,
    cost,
    last_sale_date,

    DATEDIFF(MONTH, last_sale_date, GETDATE()) AS recency_in_months,

    CASE
        WHEN total_sales > 50000 THEN 'High-Performer'
        WHEN total_sales >= 10000 THEN 'Mid-Range'
        ELSE 'Low-Performer'
    END AS product_segment,

    lifespan,
    total_orders,
    total_sales,
    total_quantity,
    total_customers,

```



avg_selling_price,

-- Average Order Revenue (AOR)
CASE

WHEN total_orders = 0 THEN 0
ELSE total_sales / total_orders
END AS avg_order_revenue,

-- Average Monthly Revenue
CASE

WHEN lifespan = 0 THEN total_sales
ELSE total_sales / lifespan
END AS avg_monthly_revenue

FROM product_aggregations;

select * from gold.report_products

#	Results	Messages	product_key	product_name	category	subcategory	cost	last_sale_date	recency_in_months	product_segment	lifespan	total_orders	total_sales	total_quantity	total_customers	avg_selling_price	avg_order_revenue	avg_monthly_revenue
1	3		Mountain-100 Black- 38	Bikes	Mountain Bikes	1898	2011-12-27	175		High-Performer	11	49	165375	49	49	3375	3375	15034
2	4		Mountain-100 Black- 42	Bikes	Mountain Bikes	1898	2011-12-27	175		High-Performer	11	45	151875	45	45	3375	3375	13806
3	5		Mountain-100 Black- 44	Bikes	Mountain Bikes	1898	2011-12-21	175		High-Performer	11	60	202500	60	60	3375	3375	18409
4	6		Mountain-100 Black- 48	Bikes	Mountain Bikes	1898	2011-12-26	175		High-Performer	12	57	192375	57	57	3375	3375	16031
5	7		Mountain-100 Silver- 38	Bikes	Mountain Bikes	1912	2011-12-22	175		High-Performer	12	58	197200	58	58	3400	3400	16433
6	8		Mountain-100 Silver- 42	Bikes	Mountain Bikes	1912	2011-12-28	175		High-Performer	11	42	142800	42	42	3400	3400	12981
7	9		Mountain-100 Silver- 44	Bikes	Mountain Bikes	1912	2011-12-12	175		High-Performer	12	49	166600	49	49	3400	3400	13883
8	10		Mountain-100 Silver- 48	Bikes	Mountain Bikes	1912	2011-12-23	175		High-Performer	11	36	122400	36	36	3400	3400	11127
9	16		Road-150 Red- 44	Bikes	Road Bikes	2171	2011-12-28	175		High-Performer	12	281	1005418	281	281	3578	3578	83784
10	17		Road-150 Red- 48	Bikes	Road Bikes	2171	2011-12-28	175		High-Performer	12	337	1205786	337	337	3578	3578	100482
11	18		Road-150 Red- 52	Bikes	Road Bikes	2171	2011-12-27	175		High-Performer	12	302	1080556	302	302	3578	3578	90046
12	19		Road-150 Red- 56	Bikes	Road Bikes	2171	2011-12-27	175		High-Performer	12	295	1055510	295	295	3578	3578	87959
13	20		Road-150 Red- 62	Bikes	Road Bikes	2171	2011-12-28	175		High-Performer	12	336	1202208	336	336	3578	3578	100184
14	36		Road-650 Black- 44	Bikes	Road Bikes	487	2012-12-26	163		Mid-Range	23	63	47565	63	63	755	755	2068
15	37		Road-650 Black- 48	Bikes	Road Bikes	487	2012-12-25	163		Mid-Range	21	60	45552	60	60	759.2	759	2169
16	38		Road-650 Black- 52	Bikes	Road Bikes	487	2012-12-19	163		High-Performer	23	89	66915	89	89	751.9	751	2909
17	39		Road-650 Black- 58	Bikes	Road Bikes	487	2012-12-18	163		High-Performer	23	76	57996	76	76	763.1	763	2521
18	40		Road-650 Black- 60	Bikes	Road Bikes	487	2012-12-12	163		High-Performer	22	76	57156	76	76	752.1	752	2598
19	41		Road-650 Black- 62	Bikes	Road Bikes	487	2012-12-18	163		Mid-Range	24	65	49047	65	65	754.6	754	2043
20	42		Road-650 Red- 44	Bikes	Road Bikes	487	2012-12-25	163		High-Performer	23	72	54528	72	72	757.3	757	2370
21	43		Road-650 Red- 48	Bikes	Road Bikes	487	2012-12-27	163		High-Performer	23	88	66720	88	88	758.2	758	2900
22	44		Road-650 Red- 52	Bikes	Road Bikes	487	2012-12-21	163		Mid-Range	24	61	46083	61	61	755.5	755	1920
23	45		Road-650 Red- 58	Bikes	Road Bikes	487	2012-12-18	163		High-Performer	22	74	56346	74	74	761.4	761	2561
24	46		Road-650 Red- 60	Bikes	Road Bikes	487	2012-12-23	163		Mid-Range	23	53	40071	53	53	756.1	756	1742
25	47		Road-650 Red- 62	Bikes	Road Bikes	487	2012-12-05	163		High-Performer	23	75	57381	75	75	765.1	765	2494
26	48		Road-250 Red- 44	Bikes	Road Bikes	1519	2012-12-25	163		High-Performer	12	144	351792	144	144	2443	2443	29316
27	49		Road-250 Red- 48	Bikes	Road Bikes	1519	2012-12-24	163		High-Performer	12	162	395766	162	162	2443	2443	32980
28	50		Road-250 Red- 52	Bikes	Road Bikes	1519	2012-12-25	163		High-Performer	12	133	324919	133	133	2443	2443	27076
29	104		Mountain Bottle Cage	Access...	Bottles and C...	4	2014-01-28	150		Mid-Range	13	2025	20340	2034	2004	10	10	1564

Query executed successfully. LK9MEEMA\SQLEXPRESS (17.0 RTM) LK9MEEMA\jenovo (76) DataWarehouseAnalytics 00:00:00 Row: 1, Col: 1 130 rows

2.7.2 Key Observations

- Product master data and sales transactions were consolidated into a single analytical report.
- Products were segmented into High-Performer, Mid-Range, and Low-Performer categories based on total sales revenue.
- Product-level metrics provide a comprehensive view of sales performance, customer reach, and product demand.



<https://www.linkedin.com/in/saideeppallela/>

- Key Performance Indicators (KPIs), including Recency, Average Order Revenue (AOR), Average Monthly Revenue, and Average Selling Price, help evaluate product performance and revenue trends.
- The report supports inventory planning, pricing optimization, product portfolio analysis, and strategic business decision-making.
- Creating the report as a SQL Server View improves data reusability and enables seamless integration with reporting tools such as Power BI and Excel.

Project Conclusion:-

- This project demonstrates my ability to apply Microsoft SQL Server to solve real-world business problems using structured data analysis and reporting. Throughout the project, I explored relational data, analyzed customer and product performance, implemented advanced SQL techniques, and developed reusable analytical reports that transform raw transactional data into meaningful business insights.
- By applying SQL concepts such as joins, aggregate functions, Common Table Expressions (CTEs), window functions, CASE expressions, and Views, I built scalable reporting solutions that support business decision-making and Business Intelligence reporting.
- This project reflects my ability to work with relational databases, analyze business data, develop performance metrics, and create business-ready SQL solutions aligned with the responsibilities of a Data Analyst.

Core Skills Demonstrated:-

- Microsoft SQL Server (T-SQL)
- Exploratory Data Analysis (EDA)
- SQL Joins & Data Modeling
- Aggregate & Window Functions
- Common Table Expressions (CTEs)
- CASE Expressions
- Customer & Product Analytics
- KPI Development
- SQL View Creation
- Business Reporting & Analytical Thinking

